Gestion des Processus, Cycle de Vie et Signaux POSIX sous Unix/Linux

Thème principal : Systèmes d'exploitation : Modèle de processus, multitâche préemptif et communication inter-processus

Synthèse exécutive : Ce cours pose les bases fondamentales de la gestion de l'exécution au sein d'un système d'exploitation moderne de type Unix/Linux. Il explicite la métamorphose d'un programme statique sur disque en un processus dynamique en mémoire RAM, le rôle de l'ordonnanceur dans le multitâche préemptif, la structure arborescente des processus (PID, PPID) issue de la racine systemd/init (PID 1), ainsi que le mécanisme de communication asynchrone représenté par les signaux POSIX (notamment SIGINT, SIGTERM et SIGKILL).

1. Distinctions Conceptuelles : Du Programme Statique au Processus Dynamique

Un système d'exploitation doit gérer les ressources matérielles de façon équitable et sécurisée. La différence entre un programme et un processus est primordiale : le programme n'est qu'un ensemble passif d'instructions codées stocké sur disque. Dès que l'utilisateur ou le système demande son exécution, le noyau (kernel) charge le code en mémoire vive (RAM), lui attribue un espace d'adressage virtuel, initialise ses registres et crée une structure de données interne appelée PCB (Process Control Block). Le processus prend alors vie. Dans un environnement multitâche préemptif, le processeur passe à très haute fréquence d'un processus à un autre grâce à l'ordonnanceur (scheduler). Chaque processus possède des attributs uniques : - PID (Process ID) : Identifiant numérique unique attribué par le système. - PPID (Parent Process ID) : Identifiant du processus parent ayant créé ce processus fils via fork(). - PCB (Process Control Block) : Structure conservant les pointeurs mémoire, la priorité, l'identifiant de l'utilisateur (UID) et les descripteurs de fichiers ouverts. Le processus évolue à travers trois états principaux gérés par le noyau : 1. Prêt (Ready) : Le processus dispose de toutes les ressources nécessaires et attend que l'ordonnanceur lui donne accès au CPU. 2. Élu (Running) : Le processus est directement exécuté par un cœur du processeur. 3. Bloqué (Waiting/Blocked) : Le processus est mis en attente d'un événement externe (ex: lecture de disque, saisie clavier, paquet réseau) et libère la CPU pour les autres processus.

- Programme : Fichier binaire ou script passif résidant sur un stockage secondaire (SSD/HDD).
- Processus : Instance dynamique en cours d'exécution occupant de la mémoire RAM et des registres processeur.
- Multitâche Préemptif : Mécanisme par lequel le noyau alloue périodiquement des quanta de temps CPU à chaque processus pour simuler une exécution parallèle.
- Prédécesseurs et Généalogie : Duplication par l'appel système fork() puis remplacement de l'espace adressable par exec(). Tout processus descend du processus racine systemd ou init (PID 1).

Cas concret enseignant : *L'analogie de la cuisine : Le programme est un livre de recettes de cuisine fermé sur une étagère (un fichier passif stocké sur le disque dur). Le processus est le cuisinier en train d'exécuter activement la recette dans la cuisine : il occupe de l'espace sur le plan de travail (mémoire RAM), utilise des ustensiles et son énergie (temps CPU), et interagit avec son environnement en temps réel.*

2. Contrôle d'Exécution et Communication Inter-Processus : Les Signaux POSIX

Les signaux constituent le moyen le plus simple et le plus ancien de communication inter-processus (IPC) sous Unix. Un signal est une notification asynchrone : à la réception du signal, le noyau interrompt le fil d'exécution normal du processus pour appeler un gestionnaire de signal (signal handler). Le développeur d'un programme peut programmer trois comportements face à un signal : 1. L'intercepter (catch) : Exécuter un bloc de code spécifique pour réagir à l'événement. 2. L'ignorer (ignore) : Ne rien faire et poursuivre l'exécution. 3. Laisser le comportement par défaut (default) : Souvent la terminaison du processus. Seules deux exceptions absolues ne peuvent être ni interceptées ni ignorées : SIGKILL (9) et SIGSTOP (19). Récapitulatif des signaux essentiels : - SIGINT (Signal 2) : Interruption déclenchée dans le terminal par 'Ctrl + C'. Utilisé pour arrêter une tâche interactive. - SIGTERM (Signal

15) : Signal d'arrêt propre envoyé par la commande 'kill '. Il laisse au processus le temps de fermer ses fichiers, sauvegarder son état et fermer ses connexions réseau. - SIGKILL (Signal 9) : Ordre d'arrêt absolu. Le noyau détruit le processus immédiatement sans consulter son code. - SIGSTOP (Signal 19) et SIGCONT (Signal 18) : Utilisés pour suspendre (Ctrl + Z) et reprendre ('fg' / 'bg') une tâche.

- Signal POSIX : Interruption logicielle envoyée à un processus pour l'informer d'un événement ou lui demander d'exécuter une action.
- Gestionnaire de Signal (Signal Handler) : Routine de code exécutée automatiquement à la réception d'un signal.
- SIGINT (2) : Signal d'interruption utilisateur envoyé généralement via la combinaison Ctrl+C.
- SIGTERM (15) : Signal de terminaison propre envoyé par défaut par la commande kill.
- SIGKILL (9) : Signal de destruction immédiate et inconditionnelle, directement géré par le noyau.

Cas concret enseignant : *L'analogie de l'évacuation du bâtiment : SIGTERM (15) est le gardien d'immeuble qui frappe poliment à votre porte pour vous demander de ranger vos affaires, éteindre les lumières et sortir du bâtiment (fermeture propre et ordonnée). SIGKILL (9) est l'intervention des forces de sécurité qui défoncent la porte et vous expulsent instantanément sans vous laisser le temps de sauvegarder vos travaux.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la commande 'kill 9 1234' renvoie-t-elle une erreur alors que 'kill -9 1234' fonctionne correctement ?
Réponse du professeur : La commande kill attend que le numéro du signal soit précédé d'un tiret (ex: -9 ou -SIGKILL). Si vous tapez 'kill 9 1234', Linux comprend que vous voulez envoyer le signal par défaut (SIGTERM 15) à deux processus distincts : le PID 9 et le PID 1234. Comme le PID 9 n'existe probablement pas ou appartient au noyau, la commande renvoie une erreur de permission ou d'inexistence.

Question étudiant : Qu'est-ce qu'un processus dit 'Zombie' et comment le détruit-on avec la commande kill -9 ?
Réponse du professeur : Un processus Zombie est un processus enfant dont l'exécution est terminée, mais dont le processus parent n'a pas encore lu le code de retour via l'appel système wait(). Le processus zombie est déjà mort : il ne consomme plus ni mémoire RAM ni processeur, il n'occupe qu'une ligne dans la table des processus. On NE PEUT PAS tuer un zombie avec 'kill -9' car il est DÉJÀ mort ! Pour le faire disparaître, il faut forcer le processus parent à lire son code de sortie, ou tuer le processus parent pour que le processus Zombie soit adopté par 'systemd' (PID 1) qui le nettoiera immédiatement.

■ Exercice & Démonstration d'Application

Énoncé : Dans un terminal Bash sous Linux, un utilisateur exécute la séquence de commandes suivante : 1. sleep 600 & 2. ps -ef | grep sleep 3. kill -15 Expliquez le rôle du symbole '&', détaillez le comportement exact de la commande 'kill -15' et indiquez le résultat d'un contrôle ultérieur de la table des processus.

Résolution pas à pas :
Étape 1 : 'sleep 600 &' lance la commande d'attente de 600 secondes en arrière-plan (background) grâce au symbole '&'. Le shell rend immédiatement la main à l'utilisateur et affiche le numéro de job ainsi que le PID attribué au processus (exemple : [1] 4521). Étape 2 : 'ps -ef | grep sleep' liste tous les processus du système et filtre les lignes contenant la chaîne 'sleep'. On y identifie le PID du processus (ex: 4521) ainsi que son PPID (qui correspond au PID du shell Bash). Étape 3 : 'kill -15 4521' envoie le signal SIGTERM (Signal 15) au processus 4521. Comme le programme sleep utilise le comportement par défaut pour SIGTERM, il intercepte le signal et se termine proprement.

Résultat : Le processus 'sleep 600' est arrêté proprement. Un second contrôle exécuté avec 'ps -ef | grep sleep' confirme que le processus a été supprimé de la table des processus actifs du noyau.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le nom et le PID du tout premier processus utilisateur démarré par le noyau Linux ?

R : Le tout premier processus utilisateur est 'systemd' (ou 'init' sur les anciens systèmes Unix), et son PID est toujours le numéro 1.

Q : Quel signal POSIX est envoyé au processus en premier plan lorsque l'utilisateur effectue le raccourci 'Ctrl + C' dans un terminal ?

R : La combinaison 'Ctrl + C' envoie le signal SIGINT (Signal numéro 2) au processus.

Q : Pourquoi le signal SIGKILL (numéro 9) ne peut-il jamais être intercepté ni ignoré par un programme informatique ?

R : Parce que le signal SIGKILL n'est pas transmis au gestionnaire de signaux du programme : il est intercepté et exécuté directement et exclusivement par le noyau (kernel), qui détruit sur-le-champ l'espace mémoire et le contexte du processus.

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765